For your viva, you should be able to explain the code **block by block**, not line by line. Here's how you can explain each section confidently.

---

1. Pin Declaration

```
const int sensorPin = A1;
const int outputPin = 7;
const int buttonPin = 4;
```

Viva Answer

- **A1** receives the analog signal from the ACS712 current sensor.
- **Pin 7** controls the MOC3021 optocoupler, which in turn triggers the TRIAC.
- **Pin 4** is connected to the reset push button.

---

2. Constants

```
const float sensitivity = 0.066;
const float tripThreshold = 0.5;
```

Viva Answer

- The ACS712-30A sensor has a sensitivity of **66 mV/A**.
- The protection threshold is set to **0.5 A**. If the measured RMS current exceeds this value, the protection system trips.

---

3. Trip State

```
bool tripState = false;
```

Viva Answer

This variable stores whether the system is in the protection state.

- **false** = Normal operation
- **true** = Overcurrent detected and protection activated

It acts as a **latch**, meaning the system stays tripped until the reset button is pressed.

---

4. Setup Function

```
Serial.begin(9600);
```

Viva Answer

Starts serial communication so the Arduino can continuously display the RMS current on the Serial Monitor.

```
pinMode(outputPin, OUTPUT);
pinMode(buttonPin, INPUT_PULLUP);
```

Viva Answer

Pin 7 is configured as an output.

Pin 4 uses the Arduino's internal pull-up resistor, so the button normally reads HIGH and becomes LOW when pressed.

```
digitalWrite(outputPin, HIGH);
```

Viva Answer

Initially Pin 7 is HIGH.

This turns ON the MOC3021, which triggers the TRIAC and supplies power to the AC load.

---

5. Number of Samples

```
const int samples = 100;
```

Viva Answer

The Arduino takes **100 ADC samples** for each measurement cycle.

Multiple samples improve the accuracy of RMS current calculation while still providing a reasonably fast response.

---

6. Dynamic Offset Calculation

```
adcSum += analogRead(sensorPin);
```

then

```
float adcOffset = adcSum / (float)samples;
```

Viva Answer

The ACS712 output is centered around approximately **2.5 V** when there is no current.

Instead of assuming exactly 2.5 V, the Arduino calculates the average sensor output. This average becomes the reference (offset), helping compensate for noise and small voltage variations.

---

7. RMS Current Calculation

```
float voltage =
((adc-adcOffset)*5.0)/1023.0;
```

Viva Answer

The Arduino converts the ADC reading into voltage.

Subtracting the offset removes the DC component, leaving only the AC waveform.

```
float current =
voltage/sensitivity;
```

Viva Answer

Using the ACS712 sensitivity of **66 mV/A**, the voltage is converted into instantaneous current.

```
sumSquares += current*current;
```

Viva Answer

Each instantaneous current value is squared.

Squaring removes the effect of positive and negative half cycles and prepares the values for RMS calculation.

```
sqrt(sumSquares/samples);
```

Viva Answer

Finally, the square root of the average squared current is taken to obtain the **RMS current**, which is the effective AC current.

---

8. Low-pass Filter

```
filteredCurrent =
0.9*filteredCurrent
+
0.1*rmsCurrent;
```

Viva Answer

This is a simple digital low-pass filter.

It smooths the RMS readings by reducing random fluctuations and electrical noise, resulting in a more stable current display.

---

9. Serial Monitor

```
Serial.print(...)
```

Viva Answer

The Arduino continuously prints the filtered RMS current for monitoring and testing purposes.

---

10. Trip Logic

```
if(!tripState &&
filteredCurrent>tripThreshold)
```

Viva Answer

If the current exceeds **0.5 A**, and the system is not already tripped, the protection is activated.

The condition `!tripState` prevents repeated triggering after the first trip.

---

11. Reset Button

```
if(digitalRead(buttonPin)==LOW)
```

Viva Answer

When the push button is pressed, Pin 4 becomes LOW.

The trip latch is cleared and the protection system returns to normal operation.

---

12. Output Control

```
if(tripState)
digitalWrite(outputPin,LOW);
else
digitalWrite(outputPin,HIGH);
```

Viva Answer

- **Normal operation:** Pin 7 remains HIGH, so the MOC3021 continues triggering the TRIAC and the load stays ON.
- **Overcurrent:** Pin 7 goes LOW, the MOC3021 stops triggering the TRIAC, and the load is disconnected for protection.

Most Important Viva Question

Examiner: *Why did you calculate RMS current instead of using a single ADC reading?*

Answer:

> The load operates on **220 V, 50 Hz AC**, so the current changes continuously between positive and negative values. A single ADC reading only gives the instantaneous current at one point in the waveform, which is not representative of the actual load current. By sampling multiple points and calculating the RMS value, the Arduino obtains the effective current, which is the correct quantity for overcurrent protection.

This is one of the strongest answers you can give because it demonstrates you understand both the code and the underlying electrical principle.